

Farmer Marketplace: Phase 1 to 3 Build Audit

Audited on 2026-10-03 against *FARMER_MARKETPLACE_BUILD_GUIDE.md*, using the code, the migrations, the 12 passing tests and a live run of the API.

In short

Every feature the guide asks for in Phases 1 to 3 works. The gaps are in how the code is organised, how much of it is tested, and a few edge cases and safety issues. The core rule holds: the marketplace runs with no Solana, AI or x402 code.

Phase	Met	Partial	Missing	Score
Phase 1: foundation	14	4	5	70%
Phase 2: matching	14	2	0	94%
Phase 3: pooled transport	15	3	0	92%

Score = (Met + half of Partial) divided by the requirements checked. Each requirement is listed with its evidence from page 2 onward.

What is built well

- Phase 2 is essentially complete: radius search, an explainable score, multi-farmer allocation, saved match plans, a match-plan screen, and race-safe grouped orders.
- Phase 3 works end to end: pools, vehicle quotes, confirmation, and the pickup and delivery lifecycle, with a routing interface and fallback.
- Authorization is enforced on the backend, and the order state machine lives in one place.

Main gaps

- **Safety (high):** POST /demo/seed needs no login, is on by default, and wipes every marketplace table.
- **Structure:** no repository layer, no per-module models or schemas, no payments module, and the guide's folder layout is only partly followed.
- **Tests:** nothing covers auth, listing CRUD, permissions or the HTTP API. All 12 tests call services directly.
- **Pooling edge cases:** a proposed pool can never be rejected, so its orders stay locked. Direct orders have no destination and can never be pooled. Delivery windows cannot be set in the UI.
- **Not yet built:** domain events, `external_transactions`, and interfaces for payments, notifications and attestations.

Before starting Phase 4

Disable the demo seed outside local use, add a way to dissolve a proposed pool, add HTTP-level tests, and add `external_transactions` with an in-process events module. The full ranked list is in section 6.

Detailed audit

1. Method

Every requirement under a phase heading in the guide (its Build list, Minimum API, Definition of done, architecture rules and its Testing Strategy for that phase) was checked against the code. Evidence is a file and endpoint or test name, so each row can be re-verified. Status values: **Met** (works as the guide describes), **Partial** (works but incomplete or different), **Missing** (absent). File paths are relative to `api/app` unless they start with `tests/` or `migrations/`.

Evidence base: 12 tests pass (`pytest api/tests`, 6 in `test_phase1.py` and 6 in `test_phase3.py`); 2 migrations (`0001_marketplace`, `0002_logistics`) apply cleanly; 34 route handlers across 8 route files; the pooling and delivery endpoints were exercised over HTTP against PostgreSQL/PostGIS.

2. Phase 1: Marketplace foundation

Requirement	Status	Evidence
Farmer register/login	Met	<code>auth/routes.py</code> : POST <code>/auth/register</code> , <code>/auth/login</code> (PBKDF2-SHA256, 120k rounds)
Farmer profile	Met	<code>users/routes.py</code> : POST <code>/farmer-profile</code> , GET <code>/farmer-profile/{id}</code>
Create farm + location	Met	<code>farms/routes.py</code> POST <code>/farms</code> ; stored as lat/lon and PostGIS geography (<code>store.py</code> <code>Farm.location</code>)
Create / edit / deactivate listings	Met	<code>catalog/routes.py</code> POST <code>/lots</code> , PATCH <code>/lots/{id}</code> with active flag; ownership checked
Buyer register/login	Met	Same auth routes, default role buyer
Browse products, filter by product and price	Met	GET <code>/products</code> and <code>/lots</code> with product and max_price filters (active lots only)
View farmer / listing	Partial	GET <code>/lots/{id}</code> and GET <code>/farmer-profile/{id}</code> exist, but the lot view has no farm name or location, and GET <code>/farms</code> is farmer-only
Create order / request	Met	POST <code>/orders</code> , POST <code>/buyer-requests</code> (buyer role)
Admin: moderate users and listings	Met	<code>admin/routes.py</code> PATCH <code>/admin/users/{id}</code> , <code>/admin/lots/{id}</code> (admins cannot be disabled)
Admin: view orders	Met	GET <code>/admin/orders</code>
Minimum API: all 11 endpoints	Met	All 11 listed endpoints exist; <code>/products</code> is an alias of <code>/lots</code>
Definition of done: list, discover, order, accept, complete	Met	<code>test_marketplace_flow</code> , plus the HTTP demo flow
Single service owns order state transitions	Met	<code>orders/service.py</code> <code>transition()</code> and <code>VALID_TRANSITIONS</code>
Backend validates authorization	Met	<code>shared/auth.py</code> <code>require_role</code> on every write route; owner checks on lots, farms, orders
Module layout (auth, users, farms, catalog, orders, payments, shared)	Partial	All present except payments/
Per-module models / schemas / repository / service / routes	Missing	Schemas are inline in <code>routes.py</code> , all models live in <code>shared/store.py</code> , there are no <code>repository.py</code> files
Route -> Service -> Repository -> Database	Partial	Only orders, matching and logistics have a service; no repositories anywhere
No business rules in routes	Partial	Ownership checks and field updates in catalog, farms and users routes are in the handlers
Test: authentication	Missing	No test registers or logs in
Test: listing CRUD	Missing	No test creates, edits or deactivates a lot

Requirement	Status	Evidence
Test: order states	Met	test_marketplace_flow, test_pool_confirm_and_delivery_flow (valid path and one invalid transition)
Test: permissions	Missing	Only confirm_pool's 403 is tested; no role or ownership tests on routes
Test: API integration	Missing	Tests call services, not HTTP; starlette TestClient needs httpx2, which is not installed

3. Phase 2: Location matching and aggregation

Requirement	Status	Evidence
PostGIS geographic points, radius query	Met	Geography(POINT, 4326) with GiST index; engine.py uses ST_DWithin and ST_Distance
Product filter	Met	Case-insensitive product match in MatchingEngine.match
Availability filter	Met	Lot.active is true; a lot goes inactive when its quantity reaches 0
Price filter	Met	price_per_kg <= request.max_price_per_kg when set
Explainable ranking score (distance, price, risk), lower is better	Met	0.5 distance + 0.3 price + 0.2 fulfilment risk; every term returned in the API and shown in the UI
No machine learning	Met	Deterministic arithmetic only
Do not use DFS for shortest path; PostGIS/Haversine for proximity	Met	PostGIS for search, Haversine only inside the routing mock
Dijkstra/A* via a routing engine for road distance	Partial	Not used in buyer search (straight-line only); the road factor appears only in Phase 3 mock routing
Supply aggregation across farmers (150+100+150 style)	Met	Greedy fill; demo plan is 60 kg + 40 kg
MatchPlan / Allocation object before the final order	Met	match_plans and allocations tables; save_match_plan()
Final order only from a complete plan; stock revalidated	Met	create_grouped_orders locks the request and lots (FOR UPDATE), returns 409 if partial or already ordered
MatchingEngine behind an interface	Partial	A concrete class that takes a DB session; no Protocol, so swapping engines needs a small refactor
Match-plan UI (Milestone B)	Met	page.tsx match card: rank, farmer, distance, cost, score breakdown, shortfall, total
Test: radius query	Met	test_matching_uses_radius_and_aggregates (radius 0.1 km returns nothing)
Test: deterministic ranking	Met	test_ranking_is_deterministic_and_best_first
Test: partial allocation and insufficient supply	Met	test_insufficient_supply_is_partial_and_not_orderable

4. Phase 3: Pooled transportation

Requirement	Status	Evidence
Transporter entity	Partial	Role 'transporter' on users; no transporter profile table (guide lists TransporterProfile as later)
Vehicle	Met	vehicles table, POST/GET /vehicles
Shipment and ShipmentStop	Met	shipments (one per pool) and shipment_stops (ordered pickups then deliveries)
TransportPool and TransportQuote	Met	transport_pools, transport_quotes (one per vehicle that fits the load)
Group by destination area	Met	Within 5 km of the cluster's first destination (pooling.py DEST_RADIUS_KM)
Group by delivery window	Partial	Window intersection is enforced in the algorithm, but the UI cannot set windows, so it is never exercised in the app

Requirement	Status	Evidence
Group by pickup proximity	Met	Within 30 km of the cluster's first pickup
Respect vehicle capacity	Met	Cluster weight is capped at the largest active vehicle (1000 kg if none)
Compare separate vs pooled cost; pool only if it saves enough	Met	separate_cost vs pooled_cost; pooled <= 90% of separate
Simple heuristic first, no global VRP solver	Met	Greedy single pass with nearest-neighbour stop order
Optimization kept outside orders/	Met	Lives in api/app/logistics (the guide's folder name is services/routing)
RoutingProvider interface: route() and estimate()	Met	routing.py Protocol
Implementations: OSRM, GraphHopper, Mock	Partial	MockRoutingProvider and FallbackRoutingProvider only; no OSRM or GraphHopper adapter
Order states READY_FOR_PICKUP, IN_TRANSIT, DELIVERED	Met	VALID_TRANSITIONS plus /ready, /pickup, /deliver; pickup and delivery limited to the confirmed transporter
Test: capacity	Met	test_capacity_limits_pooling
Test: incompatible time windows	Met	test_far_destinations_and_incompatible_windows_do_not_pool
Test: pooling savings calculation	Met	test_compatible_orders_pool_and_save_money, test_pool_skipped_when_savings_too_small
Test: route-provider failure fallback	Met	test_routing_failure_falls_back_to_mock

5. Cross-cutting checks

Architecture rules (guide section 3) and the recommended database skeleton (section 4). These are not counted in the phase scores.

Check	Status	Evidence
Rule 1: domain logic independent of infrastructure	Met	No external service (payments, chain, maps) is imported by orders, matching or logistics
Rule 2: adapters for external services	Partial	Only routing has an interface. Payments, blockchain, notifications, object storage, AI and x402 have none; the guide says to define these boundaries from Phase 1
Rule 3: PostgreSQL is the source of truth	Met	All state is in PostgreSQL/PostGIS
Rule 4: events for cross-module actions	Missing	No domain events; transitions do not notify other modules
Rule 5: no premature microservices	Met	One API, one web app, one database
Pin major dependencies	Met	api/requirements.txt uses bounded ranges; apps/web/package.json pins versions

Table(s)	Status	Evidence
users, farmer_profiles, farms, produce_lots (lots), buyer_requests, orders	Met	migration 0001
match_plans, match_allocations	Met	match_plans and allocations
shipments, shipment_stops, transport_pools	Met	migration 0002, plus vehicles and transport_quotes
buyer_profiles, products, order_items	Missing	A buyer has no profile, products are free text on lots, and an order covers a single lot

Table(s)	Status	Evidence
payments, external_transactions	Missing	The guide says to create external_transactions early so providers stay generic
transporter_profiles	Missing	Role only
attestations, reputation_events	Missing	Phase 4, expected to be absent

6. Defects and risks

Ordered by severity. Each row names the code location, a concrete failure and a fix.

ID	Severity	Issue	Where	Failure	Fix
D1	High	Demo seed is open and destructive	demo.py:15 defaults ENABLE_DEMO_SEED to on; shared/demo.py:13 reset_demo_data deletes every marketplace table; the route needs no login.	Anyone who can reach the API calls POST /demo/seed and erases all users, lots and orders.	Default the flag to off, require admin auth, or serve the route only in a dev profile.
D2	Medium	A proposed pool cannot be rejected	logistics/service.py:20 and :40 stamp order.pool_id on proposal; no route clears it.	A transporter proposes a pool and never confirms. Those orders are excluded from future pooling forever.	Add a dissolve endpoint that clears pool_id and deletes the pool, and expire stale PROPOSED pools.
D3	Medium	No order cancellation, and pools ignore status changes	orders/service.py:11-13 allows CANCELLED but no route calls it; confirm_pool does not re-check order status.	A cancelled order would still ride on a confirmed shipment.	Add a cancel endpoint that also removes the order from an unconfirmed pool and blocks confirmation if any order is no longer eligible.
D4	Medium	Direct orders can never be pooled	orders/service.py:39 copies a destination only when an order comes from a buyer request; POST /orders has no destination.	A buyer who orders a lot directly gets an order that no pool will ever include.	Accept an optional destination and window on POST /orders, or document that only request-based orders ship.
D5	Medium	Auth tokens never expire	shared/auth.py looks the token up by id only; Token.created_at (store.py:31) is never checked; tokens are stored in plaintext.	A leaked token works forever, and a database leak exposes live sessions.	Add an expiry check, store a hash of the token, and add a logout route.
D6	Low	Delivery windows cannot be set in the UI	BuyerRequestCreate accepts window_start and window_end (orders/routes.py) but the request form has no fields for them.	Window-based pooling cannot be demonstrated through the app.	Add two datetime inputs to the request form.
D7	Low	Demo shortcut skips delivery	orders/service.py:12 allows ACCEPTED to COMPLETED.	An order can complete without ever being picked up or delivered, which Phase 4 evidence and later payments should not allow.	Remove the shortcut when the delivery flow becomes the default, and update the Phase 1 test.
D8	Low	Lot view hides the farm	catalog/routes.py lot_view returns farm_id only.	A buyer cannot see where a lot is from without a farmer-only call.	Return farm name and coordinates in the lot view.

7. Test inventory

All 12 tests pass. They run against the real PostgreSQL/PostGIS database and roll back after each test. None goes through HTTP.

Test	File	What it proves
test_marketplace_flow	test_phase1.py	Direct order, accept, complete; lot goes inactive when sold out
test_demo_seed_is_persisted	test_phase1.py	Seed data survives a commit (5 users, accepted order, open request)
test_matching_uses_radius_and_aggregates	test_phase1.py	100 kg filled by 2 farmers; scores within 0 to 1; radius 0.1 km returns none
test_complete_match_creates_grouped_orders_once	test_phase1.py	Grouped order totals 100 kg, shares a group id, second attempt returns 409
test_insufficient_supply_is_partial_and_not_orderable	test_phase1.py	Partial plan and 409 on order creation
test_ranking_is_deterministic_and_best_first	test_phase1.py	Same input gives the same ranked list, sorted by score
test_compatible_orders_pool_and_save_money	test_phase3.py	Two orders pool, over 10% saving, stops ordered pickups then deliveries
test_capacity_limits_pooling	test_phase3.py	600 kg does not pool into a 500 kg vehicle
test_far_destinations_and_incompatible_windows_do_not_pool	test_phase3.py	Far destination and non-overlapping windows stay separate
test_pool_skipped_when_savings_too_small	test_phase3.py	Pickups on opposite sides of the destination are not pooled
test_routing_failure_falls_back_to_mock	test_phase3.py	Broken provider falls back to the mock estimate
test_pool_confirm_and_delivery_flow	test_phase3.py	Propose, no double-pooling, wrong transporter 403, confirm, full lifecycle, invalid transition rejected

8. Recommended actions, in order

- **Close D1** (demo seed). Small change, removes the only high-severity risk.
- **Close D2 and D3** (dissolve pool, cancel order). Prevents orders getting stuck or shipped after cancelling.
- **Add HTTP-level tests** for auth, listing CRUD and permissions. Install httpx2 so starlette's TestClient works. This fills the five missing Phase 1 test items.
- **Add external transactions and an in-process domain events module.** Phase 4 reputation events and Phase 5 settlement both depend on them (guide Rules 2 and 4).
- **Close D4, D5 and D7** before payments exist, since they affect what counts as a completed, deliverable order.
- **Extract services and schemas** from catalog, farms, users and auth as each is next touched, rather than a single large refactor.
- **Make MatchingEngine a Protocol** when a second engine (for example OR-Tools) is actually needed.